Doc. No.:	WG21/P0062R1
Date:	2016-05-27
Reply to:	Hans-J. Boehm
Other Contributors:	JF Bastien, Peter Dimov, Hal Finkel, Paul McKenney, Michael Wong, Jeffrey Yasskin
Email:	hboehm@google.com
Target:	SG1 for now

P0062R1: When should compilers optimize atomics?

[This was revised, and the concrete wording adjusted, in response to discussion in Kona.]

Programmers often assume that atomic operations, since they involve thread communication, are not subject to compiler optimization. N4455 tries to refute that, mostly by exhibiting optimizations on atomics that are in fact legal. For example, `load(y, memory_order_`*x*`) + load(y, memory_order_`*x*`)` can be coalesced into `2 * load(y, memory_order_`*x*`)`. In many cases, such optimizations are not only allowed, but in fact desirable. For example, removing a (non-`volatile`) `atomic` load whose value is not used seems uncontroversial.

However, some optimizations on atomics are controversial, because they may actually adversely affect progress, performance, or responsiveness of the whole application. This issue was raised in a recent, rather lengthy, discussion on the c++std-parallel reflector. The purpose of this short paper is to reflect some of that discussion here. I believe that, even if we cannot say anything normative in this area, it would be very useful to clarify our intent, if we can agree what it is. Atomic optimization can easily cause an application to misbehave unacceptably if the programmer and compiler writer disagreed on our intent; I believe this is too serious a code portability issue to write this off as "quality of implementation".

To keep this discussion focused we will concentrate on transformations of atomic operations themselves, rather than mixed atomics and non-atomics.

As a simple example, consider:

```
atomic<int> progress(0);

f() {
    for (i = 0; i < 1000000; ++i) {
        // ...
        ++progress;
    }
}
```

where the ellipsis represents work that does not read or write shared memory (or where the increment is replaced by a `memory_order_relaxed operation)`, and where the variable `progress` is used by another thread to update a progress bar.

An (overly?) aggressive compiler could "optimize" this to:

```
atomic<int> progress(0);
f() {
    int my_progress = 0; // register allocated
    for (i = 0; i < 1000000; ++i) {
        // ...
        ++my_progress;
    }
    progress += my_progress;
}
```

This transformation is clearly "correct". An execution of the transformed `f()` looks to other threads exactly like an execution of the original `f()` which paused at the beginning, and then ran the entire loop so fast that none of the intermediate increments could be seen. In spite of this "correctness", the transformation completely defeats the point of the progress bar: In the transformed program it always pauses at zero and then jumps to 100%.

We can also consider less drastic transformations in which the loop is partially unrolled and the global progress counter is only updated in the outer loop, for example after every ten iterations. This would probably not interfere with the progress bar. It might be more problematic is the counter is instead read by a watchdog thread which restarts the process if it fails to observe progress. But in any case, it's usually not too different from the kind of delays normally introduced by hardware and operating systems anyway.

The only statement made by the current standard about this kind of transformation is 1.10p28:

> "An implementation should ensure that the last value (in modification order) assigned by an atomic or

synchronization operation will become visible to all other threads in a finite period of time."

This arguably doesn't restrict us here, since even a million loop iterations presumably still qualify as "a finite period of time".

Our prohibition against infinite loops (1.10p27) also makes this statement still weaker than one might expect.

```
x.store(1); while (...) { … }
```

can technically be transformed to

```
while (...) { … } x.store(1);
```

so long as the loop performs no I/O, `volatile`, or `atomic` operations, since the compiler is allowed to assume termination.

Peter Dimov points out that fairly aggressive optimizations on atomics may be highly desirable, and there seems to be little controversy that this is true in some cases. For example, atomics optimizations might eliminate repeated `shared_ptr` reference count increment and decrement operations in a loop.

Paul McKenney suggests avoiding problems as in the above progress bar example by generally also declaring each `atomic` variable as `volatile`, thus minimizing the compiler's choices. This unfortunately leads to a significantly different style, and probably worse performance for compilers that are already cautious about optimizing atomics. It also is not technically sufficient. For example, in the above progress bar example, it would still be allowable to decompose the loop into two loops, such that the second loop did nothing but repeatedly increment `progress` a million times.

My personal feeling is that we should:

1. Discourage aggressive compiler optimizations by default.
2. Possibly provide some way to specify exceptions to this rule, e.g. via an attribute.

The argument for (1) is that aggressive atomic optimizations may be extremely helpful (e.g. Peter Dimov's `shared_ptr` example), or extremely unhelpful (e.g. progress bar example above, or cases in which delayed updates effectively increase critical section length and destroy scalability). Compilers are almost never in a position to distinguish between these two with certainty. I believe that if a compiler doesn't know what it's doing, it should leave the code alone.

Peter Dimov's example provides a good argument for (2). Implementations could handle specifically `shared_ptr` this way, even without additional facilities in the standard. But that wouldn't cover e.g. intrusive reference-counting schemes, which are also justifiably common.

Conclusions from Kona:

We discussed P0062R0 in Kona. There was a fairly strong consensus that we should provide more control over aggressive atomics optimization. There was a slight majority, though not consensus, against discouraging aggressive optimizations by default. This wording follows the majority.

A wording proposal to provide increased control over optimization of atomics:

I am not deeply attached to the "brittle_atomic" name.

Add a new section near 7.6.5 [dcl.attr.deprecated]:

> **7.6.? Brittle atomic attribute [dcl.attr.brittle]**
>
> The *attribute-token* `brittle_atomic` can be used to identify statements containing atomic operations that should be less aggressively optimized.
>
> This attribute requests that implementations should limit program transformations involving calls to atomic operations textually contained in the associated statement. Timing of these atomic operations may be important to program performance or interactive program response. Such atomic operations should not be postponed or advanced appreciably more than what would already be expected from hardware optimizations.
>
> [Note: A typical example might be a conditional that uses an atomic store to release a spin lock after the then clause, but releases it before a potentially long running loop in the else clause. The implementation might otherwise merge the two atomic stores into a single one following the entire conditional. If the atomic store in the else clause is preceded by `[[brittle_atomic]]`, then the compiler should not do so. This may prevent the critical section from being substantially extended. The use of `volatile` qualifiers for this purpose may not be effective, and is discouraged. -- end note]